

Trabalho Prático 02

Questão Comparativo Ordenações

Fabiano Queiroz Guimarães Almeida¹

¹Instituto de Ciências Exatas e Informática – Universidade Pontifícia
Católica de Minas Gerais (PUC-MINAS)
Belo Horizonte, MG – Brazil

***Resumo.** Este artigo visa apresentar o número de comparações e movimentações entre elementos de arrays e o tempo de execução dos algoritmos de ordenação de Seleção, Inserção, Bolha e Quicksort. Diante disso, também busca-se analisar por meio de gráficos e discutir acima desses resultados com a finalidade de obter observações e conclusões acerca das características principais de tais métodos de ordenação em estudo na área de Ciência da Computação.*

1. Informações gerais

Todos os dados de comparações, movimentações e tempo de execução foram obtidos por meio da implementação de um código na linguagem de programação em Java (**versão: java "22.0.2" 2024-07-16**) na IDE *Visual Studio Code*. Todos os arrays usados para este trabalho foram declarados normalmente de acordo com os padrões do Java, e depois foram inicializados em duas etapas: Na primeira usou-se uma função que inicializa cada elemento da posição *y* de um array com um valor *y*, de modo que a estrutura de dados tenha elementos em ordem crescente. Já na segunda etapa, outra função troca cada elemento *y* desse array com outro elemento de uma posição aleatória (obtido pela classe *Rand*).

2. Dados Obtidos

De acordo com os gráficos apresentados abaixo, podemos ver que o número de comparações em todos os casos do algoritmo de seleção aumentava de acordo com o tamanho do vetor, enquanto o número de movimentações em relação a comparações sempre ficava baixo. Já o de inserção, as comparações aumentavam de acordo com a capacidade do vetor embora sempre abaixo do número das do algoritmo de seleção, exceto no vetor de 100.000 elementos. O algoritmo de bolha tem sempre mais movimentações do que comparações, e suas movimentações passam as comparações do seleção, embora as comparações entre o seleção e bolha são sempre iguais. Por fim, o *quicksort* apresenta movimentações e comparações baixíssimas em relação a todos os outros algoritmos.

3. Gráficos e Ampliamento das métricas de tempo

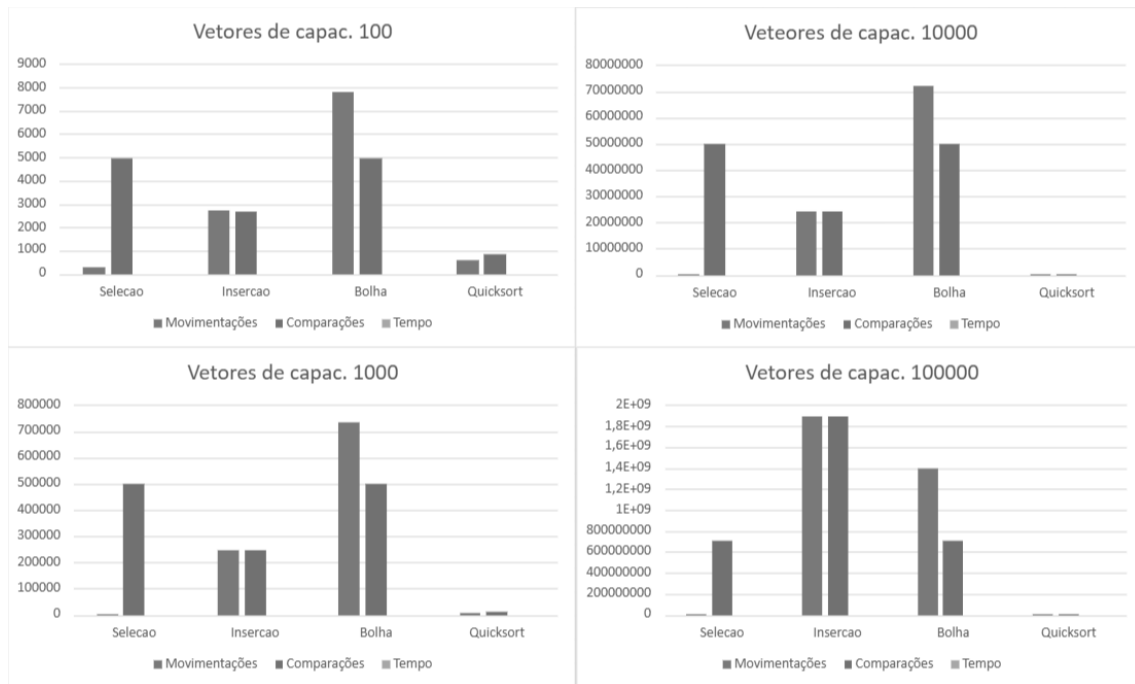


Figure 1. Dados obtidos

Vetor/Alg.	Seleção	Inserção	Bolha	<i>Quicksort</i>
100	0.0	0.0	0.001	0.0
1000	0.004	0.005	0.007	0.001
10.000	0.067	0.034	0.125	0.002
100.000	3.561	2.748	12.875	0.009

Table 1. Tempo de execução

4. Análise

De acordo com os resultados, podemos primeiramente observar a grande vantagem do *quicksort* em relação a movimentações, comparações e de tempo, visto que, embora gaste uma quantidade de memória maior que outros algoritmos como o inserção, bolha e seleção, ele é um algoritmo "*in-place*", que utiliza o vetor em si para fazer suas operações. Além disso, temos o bolha e seleção, que por fazer um grande número de comparações de elementos do vetor, tem as métricas de comparação altas, incluindo as de tempo, embora apenas o bolha tenha muitas movimentações, já que a cada elemento que é menor que seu antecessor, uma comparação é feita dentro do *loop* do bolha. Isso não acontece com as poucas movimentações do seleção, que não excede o número de espaço do vetor. Por fim, temos o inserção, que apresenta baixa comparações e movimentações em relação ao bolha e seleção até o vetor 10.000, que depois desse marca passa a ser o algoritmo com maior métrica de movimentações e comparações, provavelmente devido ao grande número de movimentações e comparações que precisa de fazer para inserir cada elemento

de um grande vetor de 100.000 elementos. Por fim, em relação ao tempo, o seleção e bolha ambos possui alta ineficiência devido, respectivamente, a alta taxa de comparações e pelo bolha, pela forma primitiva que ele constantemente realiza *swaps* e comparações incessantes.

5. Referências

Nenhuma referência substancial foi usada durante a produção desse trabalho.